

PeTTaChainer Codebase Assessment: Strengths, Weaknesses, and Repair Directions

Prepared for Ben Goertzel and the PeTTaChainer author

2026-07-04

Abstract

This note is an author-facing assessment of the checked-out PeTTaChainer codebase used in the petta-memory project. It is intentionally written as an ASCII LaTeX source file so it can be edited, diffed, mailed, and compiled with ordinary LaTeX tooling. The assessment is based on source inspection of `projects/petta-memory/repos/PeTTaChainer` at commit `e4db5cad60a39c0d3f81a07296d606af6de4d76d`, on the sibling PeTTa checkout at commit `d8d46920269ced70cd6236a5182d4d2409c1e12b`, and on the local petta-memory integration probes through commit `cda5cbe`. The short version is: PeTTaChainer has an unusually valuable semantic design for contextual, paraconsistent PLN, but the current runtime path has a serious materialization/compilation bottleneck that makes it risky as the near-term production chainer for petta-memory until the add path is instrumented and hardened.

Contents

1	Executive summary	3
2	Scope and evidence	3
2.1	Repository and commit scope	3
2.2	Integration evidence from petta-memory	4
3	Architectural overview	4
3.1	Python surface	4
3.2	MeTTa runtime layout	5
3.3	Core statement and query model	5
4	Major strengths	5
4.1	The pi-PLN semantic idea is the right one for this use case	5
4.2	The code is runnable as a package, not only as a notebook	6
4.3	The repository includes author-facing language documentation	6
4.4	There is a real validation seam	6
4.5	The chainer has both forward and backward capabilities	6
4.6	Proof and audit concerns are unusually prominent	6
4.7	Benchmark coverage is broader than usual	6
4.8	The codebase already separates semantic layers	7

5	Major weaknesses and risks	7
5.1	The public add path is brittle	7
5.2	The materializer can time out before real compilation begins	7
5.3	No supported precompiled-add or bulk-load API was found	8
5.4	Runtime dependency setup is heavy	8
5.5	Timeout protection is incomplete around adds	8
5.6	The global runtime load flag may hide lifecycle problems	8
5.7	Contextual query temporarily injects atoms into &self	8
5.8	The semantic surface is larger than the validation surface	9
5.9	Benchmarks may not isolate the failing micro-stage	9
6	Detailed observations by subsystem	9
6.1	Packaging and installation	9
6.2	Python API	9
6.3	Compilation path	9
6.4	Query runtime	10
6.5	Context generation	10
6.6	Truth-value and evidence semantics	10
6.7	Tests and verification	10
7	Minimal failing shape to hand to the author	10
8	Recommended fixes and improvements	11
8.1	Priority 1: stage-isolated compiler diagnostics	11
8.2	Priority 2: repair or replace recursive materialization	12
8.3	Priority 3: add a supported precompiled or bulk-add API	12
8.4	Priority 4: add timeout-safe add methods to the Python API	12
8.5	Priority 5: strengthen semantic validation	12
8.6	Priority 6: expose context search limits in outputs	13
9	Recommended petta-memory stance	13
10	Suggested regression tests for PeTTaChainer	13
10.1	Fast materializer identity tests	13
10.2	Stage timing tests	14
10.3	Batch-add diagnostics	14
10.4	Context projection reproducibility	14
11	Strengths and weaknesses table	14
12	Conclusion	15
A	Key source paths	15
B	Petta-memory artifacts informing this report	15

1 Executive summary

PETTaChainer is not just a toy wrapper around MeTTa. It contains several ideas that are directly aligned with the requirements of an experiential memory system: proof atoms, explicit truth values, evidence-count packets, generated local contexts, context-sensitive projection to PLN truth values, forward and backward chaining, proof audit machinery, and benchmark/verifier scripts. In particular, the context layer is a good match for pi-PLN style reasoning in which evidence is not forced into a single global probability space.

The main concern is operational rather than conceptual. In local petta-memory probes, small statements pass syntax checks and the PeTTa/SWI/Janus runtime can initialize, but the public add path reaches `compileadd`, whose first binding `materialize-stmt-lambdas` can time out even on lambda-free tiny proof atoms. The latest narrowed failing shape is a four-field wrapper with adjacent arity-two nested sibling payloads, for example a proof-like envelope containing a two-argument nested statement type beside a two-argument truth-value payload. This appears to be a generic evaluator/materializer traversal issue, not merely a petta-memory token issue or an STV-specific semantic error.

My recommendation is therefore a two-track strategy:

1. Preserve PeTTaChainer as the semantic reference implementation for pi-PLN context/evidence ideas.
2. In parallel, use a more operationally stable PLN chainer for immediate petta-memory experiments, and add PeTTaChainer-compatible evidence/context semantics around it.

For the PeTTaChainer author, the highest-leverage repair is to isolate and benchmark the `materialize-stmt-lambdas` and `mm2compile` stages on a minimal failing term, then add a supported precompiled or bulk-add API so callers can load already-normalized facts without passing every insertion through the full dynamic compilation path.

2 Scope and evidence

2.1 Repository and commit scope

This assessment covers the local checkout:

- `projects/petta-memory/repos/PETTaChainer`
- Commit: `e4db5cad60a39c0d3f81a07296d606af6de4d76d`
- Commit message: `Implement piPLN: paraconsistent contextual reasoning over PeTTaChainer`
- Sibling PeTTa dependency: `projects/petta-memory/repos/PeTTa`
- Sibling PeTTa commit: `d8d46920269ced70cd6236a5182d4d2409c1e12b`

The local checkout is clean at the time of this report. The PeTTaChainer tree contains about 98 Python, MeTTa, Markdown, and Prolog source/documentation files under the `pettachainer/` package tree, plus examples and tests.

2.2 Integration evidence from petta-memory

The petta-memory integration branch has accumulated a bounded evidence trail. The important observations are:

- SWI-Prolog 9.3.36 with Janus was installed in user space and verified.
- `PeTTa().process_metta_string('!(+ 2 3)')` returned 5.
- `pettachainer.check_stmt` accepted a simple statement of the form `(: bird_robin (Bird robin) (STV 1.0 0.99))`.
- `PeTTaChainer` construction is relatively fast in the local probe, around half a second after setup.
- `check_stmt` succeeds for generated petta-memory STV statements.
- Public add/query routes through `compileadd` or `compileadd-mine`; no public precompiled-add/cache API was found by source inspection.
- Bounded probes repeatedly time out inside the add-stage materialization path before reliable `mm2compile`, full `compileadd`, query, or contextual query gates can be claimed for petta-memory inputs.

The latest local petta-memory gate, at commit `cda5cbe`, narrows the materializer blocker to adjacent arity-two nested sibling payloads in a four-field wrapper. In the petta-memory repository, `PYTHONPATH=src python3 -m unittest discover -s tests -v` passes 116 tests, including the non-live gates that preserve the boundary between checked handoff evidence and claimed inferred beliefs.

3 Architectural overview

3.1 Python surface

The public Python package is small and useful. The top-level interface exports:

- `PeTTaChainer`
- `ContextualQueryResult`
- `check_stmt`
- `check_query`
- `get_language_spec`

The main class in `pettachainer/pettachainer.py` wraps a `PeTTa` runtime, loads `pettachainer/metta/petta`, creates a unique KB name, loads a logic config, and exposes methods for adding atoms, querying, forward chaining, and contextual querying. The wrapper also stores added atoms so a contextual query can temporarily inject them into `&self` and call the context projection heads.

3.2 MeTTa runtime layout

The runtime organization is much better than a flat script pile:

- `pettachainer/metta/petta_chainer.metta`: import root and public MeTTa heads such as `compileadd`.
- `pettachainer/metta/chainer/`: compilation, truth-value formulas, distribution formulas, forward and backward chaining, proof-store helpers, query runtime, mining, and proof-structure audit.
- `pettachainer/metta/context/`: context extraction, context generation, pure-MeTTa and Prolog beam search, and inference-control modules.
- `pettachainer/metta/logic_configs/`: selectable logic configs, currently including PLN and predicate-logic variants.
- `pettachainer/metta/piPLN_paper_explained/`: runnable examples and paper-translation cells.

3.3 Core statement and query model

The language spec uses proof atoms:

```
(: proof-id type tv)
```

Statements are inserted with:

```
!(compileadd kb (: proof-id type tv))
```

Queries use:

```
!(query steps kb (: $prf type $tv))
```

The explicit `proof-id` slot is a strength because it gives proof objects a place to live. The `tv` slot currently supports ordinary STV values and several value-distribution forms such as `PointMass`, `NatDist`, `FloatDist`, and `ParticleDist`. The distinction between truth uncertainty and value uncertainty is documented clearly in the language spec.

4 Major strengths

4.1 The pi-PLN semantic idea is the right one for this use case

The most important strength is conceptual. PeTTaChainer represents evidence as context-indexed positive and negative counts, (`EC pos neg`), and only projects these counts to STV values when a local query context demands it. This is a strong fit for experiential memory, where contradictory observations should usually remain visible rather than be prematurely averaged away.

The generated-context layer is particularly relevant. The code enumerates candidate guards from evidence-packet features, splits evidence inside/outside a guard, scores the split by conflict reduction plus a query-focus bonus and a feature penalty, then projects selected evidence to an STV. This directly supports exception handling such as birds/penguins without assuming a single universal distribution.

4.2 The code is runnable as a package, not only as a notebook

The repository has been organized as an installable Python package with bundled MeTTa runtime data. The `pyproject.toml` declares the package, package data is selected through `MANIFEST.in`, and the README gives installation instructions. This matters because many experimental reasoning repositories never cross the boundary from demo scripts to importable libraries.

The Python API is also small enough to be intelligible. A user can construct a handler, add atoms, query, forward-chain, or ask for a contextual query without learning every internal MeTTa helper first.

4.3 The repository includes author-facing language documentation

The files `pettachainer/LANGUAGE_SPEC.md` and `pettachainer/LLM_RULE_SPEC.md` are useful. They document the core proof atom syntax, rule templates, query patterns, truth/value uncertainty distinction, distribution truth values, and contextual-query idea. This is a practical strength for collaboration with LLM agents, because it constrains generated rules to a smaller and more checkable language.

4.4 There is a real validation seam

`pettachainer/pln_validator.py` provides `check_stmt` and `check_query`. They are not deep semantic validators, but they catch important shape errors: statement proof IDs must not be variables, query proof slots should be variables, and truth-value forms must be in the supported set. The local petta-memory integration used this seam successfully to validate exported STV proof statements before treating them as PLN-ready handoff inputs.

4.5 The chainer has both forward and backward capabilities

The API and tests exercise forward chaining, backward queries, seeded forward chaining from selected facts, and logic-config differences. The query runtime compiles a query into a goal plus temporary adds, injects those adds, runs the chainer, externalizes proof structure, and cleans up temporary query refs. The cleanup path is important: without it, query compilation can leave confusing state behind.

4.6 Proof and audit concerns are unusually prominent

The repository contains proof-structure audit modules, showcase verification, forensic packets, witness certificates, red-team mutation tests, and replay checks. Even if some of this is heavier than petta-memory needs immediately, the orientation is good: the codebase treats proof claims as artifacts that can be replayed and attacked, not as opaque strings printed by a demo.

4.7 Benchmark coverage is broader than usual

The benchmark suite is not limited to wall-clock timing. It includes smart dispatch, generated context control, complementary evidence, forward/backward comparisons, materialized backward queries, particle-vs-exact distributions, incident response, showcase verification, and red-team forgeries. This gives the author several places to attach regression tests after fixing the current materialization bottleneck.

4.8 The codebase already separates semantic layers

There is a meaningful separation between:

- syntax and validation,
- compilation and internal proof representation,
- forward/backward chaining,
- truth-value and distribution formulas,
- context generation and context projection,
- benchmarks and forensic verification.

This separation is imperfect, but it is far better than putting all semantics in one long interpreter file. It should make targeted repair possible.

5 Major weaknesses and risks

5.1 The public add path is brittle

The public insertion methods call `compileadd` or `compileadd-mine`. The current `compileadd` path is:

```
(= (compileadd $kb $stmt)
  (let*
    (($stmt1 (materialize-stmt-lambdas $stmt))
     ($atoms (collapse (mm2compile $kb $stmt1)))
     ($iatoms (list_to_set (map-flat internalize-proof-structure $atoms)))
     ($eatoms (map-flat externalize-proof-structure $iatoms))
     ($_index (index-source-implication $kb $stmt1))
     ($_ (collapse (foreach-flat add-to-kb $iatoms)))
     ($_ (maybe-process-on-add $kb $stmt1)))
    (superpose $eatoms)))
```

This is a large amount of work for every atom insertion. For small interactive examples it may be fine. For memory-system ingestion, batch loading, and agent-generated evidence packets, it is risky unless each stage has cheap and well-bounded behavior.

5.2 The materializer can time out before real compilation begins

The most serious observed issue is that `materialize-stmt-lambdas` can be slow or nonterminating on lambda-free expressions. Source inspection says the materializer should only evaluate a term when the expression head is `|->`; otherwise it recursively rebuilds the term:

```
(= (materialize-stmt-lambdas $term)
  (if (is-var $term) $term
    (if (is-expr $term)
      (if (== (car-atom $term) |->) (eval $term)
        (cons (materialize-stmt-lambdas (car-atom $term))
              (map-flat materialize-stmt-lambdas (cdr-atom $term)))))
      $term)))
```

However, local probes show timeouts on lambda-free proof atoms. The failure has been narrowed beyond the original tokens. Empty and one-argument nested types pass; generic two-argument nested expressions inside certain full proof/envelope contexts can block. The latest failing boundary is adjacent arity-two nested siblings in a four-field wrapper.

This means that the practical blocker is probably in PeTTa/MeTTa evaluation of this recursive traversal pattern, in interaction with list/expression structure, not in user lambda execution.

5.3 No supported precompiled-add or bulk-load API was found

Source inspection found public add methods routing through `compileadd` or `compileadd-mine`. No public API was found for adding already-compiled or already-normalized facts while preserving the intended indexes/proof semantics. This leaves integrators with bad choices:

- use `compileadd` and hit the materialization bottleneck;
- directly assert low-level facts and risk bypassing important indexes;
- keep a non-live handoff cache outside PeTTaChainer, which is safe but does not exercise PeTTaChainer inference.

A supported precompiled/bulk-add interface would make the codebase much more usable for systems that already have validated proof atoms.

5.4 Runtime dependency setup is heavy

The runtime requires PeTTa, SWI-Prolog 9.3.x, Janus, and in this environment careful environment variables such as `SWIPL_HOME`, `SWI_HOME_DIR`, `LD_LIBRARY_PATH`, and sometimes `LD_PRELOAD`. This is acceptable for a research prototype but fragile for collaborator onboarding, CI, and agent-run experiments. The repository documents the requirement, but the cost remains real.

5.5 Timeout protection is incomplete around adds

The Python `query` method has a multiprocessing timeout path when possible, but `add_atom` and `add_atoms_no_check` call PeTTa directly. If `compileadd` stalls, the caller can hang unless the caller independently runs it in a subprocess. The petta-memory probes had to add subprocess isolation outside the codebase for this reason.

5.6 The global runtime load flag may hide lifecycle problems

`LOADEDLIB` and `LOADED_LOCK` avoid repeated library loads in a Python process. This is understandable, but it means that runtime state is partly global, while each handler also creates its own KB name and PeTTa instance. The interaction between global MeTTa library load state, SWI/Janus state, spawned query workers, and repeated tests deserves careful stress testing. Bugs here could appear only after long-running agent sessions.

5.7 Contextual query temporarily injects atoms into `&self`

The contextual query path copies stored atoms into `&self`, calls the context projection machinery, and removes them in a `finally` block. The `finally` is good. But temporary mutation of a shared space is inherently more fragile than querying a separate explicit evidence space. If duplicate atoms, removal failures, concurrent calls, or interrupted runtimes occur, context projection may see stale evidence.

5.8 The semantic surface is larger than the validation surface

The validator checks top-level statement/query shape and supported TV forms. It does not fully validate rule schemas, evidence packet provenance, variable safety, context features, distribution payloads, proof IDs, or whether generated terms will be safe in downstream compiler stages. That is acceptable for a thin syntax guard, but callers should not mistake it for a semantic type checker.

5.9 Benchmarks may not isolate the failing micro-stage

The benchmark suite is broad, but the current petta-memory blocker needs very small stage-isolated tests: one term, one materializer, bounded output, bounded timeout. The existing showcases prove many positive paths, but they do not by themselves explain why a tiny lambda-free proof atom can time out inside `materialize-stmt-lambdas`. A small failing regression test should be added near the materializer itself.

6 Detailed observations by subsystem

6.1 Packaging and installation

Strengths. The package metadata is clear, the MIT license is present, the package name and project URLs are defined, and the README explains that PeTTa is a sibling dependency rather than a PyPI dependency. Bundling the MeTTa runtime as package data is the right move for a Python library.

Weaknesses. The dependency chain remains nontrivial, especially the SWI/Janus build. Continuous integration should explicitly cover clean-venv installation and a tiny smoke under the documented SWI version. If CI cannot run the full stack, the README should say exactly which smoke was last run and on what platform.

6.2 Python API

Strengths. The API is concise. `ContextualQueryResult.answers` puts the generated projection first, then ordinary proofs. Query timeouts use a fresh spawned process when possible, which avoids inheriting live SWI/Janus state into a forked child.

Weaknesses. Adds lack the same timeout isolation as queries. In addition, `add_atoms_no_check` batches all additions into a single `superpose`; if one item triggers the materializer bottleneck, the entire batch becomes hard to diagnose. A safer API would provide per-atom staged results and optional subprocess isolation.

6.3 Compilation path

Strengths. The compilation pipeline has named stages: materialization, MeTTa-to-compiled conversion, proof internalization/externalization, indexing, KB insertion, and on-add processing. This is good because it gives us stage names for profiling and repair.

Weaknesses. The stages are currently fused into a single public operation. There is no obvious public way to run them individually with trace, step budget, timeout, or return structured diagnostics. For research tooling, `compileadd-explain` or `compileadd-stages` would be very valuable.

6.4 Query runtime

Strengths. The query runtime injects temporary compiled adds, runs the chainer, externalizes proofs, pretty-prints, drops empty results, and clears temporary query additions before and after. This is the right overall pattern.

Weaknesses. The same compiled-query path depends on `mm2compileQuery` and the broader compiler stack. If the compiler is brittle for certain nested shapes, query reliability will be shape-sensitive too. The temporary ref cleanup should be stress-tested under exceptions and nested queries.

6.5 Context generation

Strengths. The context layer is the strongest part of the codebase for AGI research use. It explicitly encodes evidence packets, guard enumeration, conflict measurement, split scoring, focus bonuses, feature penalties, evidence selection, STV projection, proof terms, and minimality reports. It also has both pure-MeTTa and Prolog beam variants.

Weaknesses. The scoring constants and guard-enumeration bounds are research choices that need more explicit calibration. The current pure enumerator tops out at depth 3, while beam variants can go deeper. That is fine, but the user-facing API should make the selected search regime and its limits obvious in every result.

6.6 Truth-value and evidence semantics

Strengths. The code keeps EC counts separate from STV projection, and the language spec distinguishes truth uncertainty from value uncertainty. This is exactly the kind of distinction that prevents subtle PLN modeling errors.

Weaknesses. The mapping from raw evidence/provenance to EC counts remains the responsibility of the caller or data source. That is not a bug, but a documentation and API risk. If users infer EC counts from STV confidence without a principled rule, the pi-PLN semantics will be polluted.

6.7 Tests and verification

Strengths. The Python tests cover forward/backward behavior, contextual queries, validators, context showcases, complementary evidence, incident response, smart dispatch, and forensic verification. This is unusually rich for a research prototype.

Weaknesses. The test suite contains heavy showcase tests that may be too slow for every edit. It needs a small, fast “compiler microtests” layer that isolates specific recursive compiler/materializer shapes. The current petta-memory failing shape should become one such microtest.

7 Minimal failing shape to hand to the author

The following is not claimed as the only failing shape, but it captures the boundary found by petta-memory instrumentation.

Shapes that tend to pass quickly:

- a nested type expression alone;
- a full wrapper with an atom type;

- a full wrapper with an empty nested type expression;
- a full wrapper with a one-argument nested type expression;
- generic four-field wrappers where only one relevant nested sibling has arity two.

Shapes that can block under bounded probes:

- a four-field proof/envelope-like expression containing a two-argument nested type beside an STV-like two-argument payload;
- the generalized form where adjacent sibling payloads are both nested expressions of arity two.

A representative term family is:

```
(ProofEnvelope PayloadA (Requires TypeArgSentinel0 TypeArgSentinel1)
  (RightPayload 1.0 1.0))
```

and the petta-memory-relevant proof atom is:

```
(: b-profile-000 (Requires MemoryTarget0 PLNReadyViews) (STV 1.0 1.0))
```

The author should first test `materialize-stmt-lambdas` directly on these terms, outside the full `compileadd` path, with timing and recursion trace. If direct materialization is slow, the bug is before `mm2compile`. If it is fast in isolation but slow inside `compileadd`, then the interaction with `let*`, `collapse`, `map-flat`, or runtime evaluation context is suspect.

8 Recommended fixes and improvements

8.1 Priority 1: stage-isolated compiler diagnostics

Add a diagnostic head, or a Python helper, that runs the public add pipeline one stage at a time:

1. validate/evaluate statement;
2. materialize lambdas;
3. run `mm2compile`;
4. internalize proof structure;
5. externalize proof structure;
6. index source implication;
7. add internalized atoms to the KB;
8. run on-add processing.

Each stage should return structured information: status, elapsed time, output size, abbreviated output hash, and error/timeout. This can be implemented in Python with subprocess boundaries if MeTTa-level timeouts are unavailable.

8.2 Priority 2: repair or replace recursive materialization

The current recursive definition of `materialize-stmt-lambdas` is elegant but appears operationally dangerous. Possible repairs:

- Add a fast path: if a statement contains no `|->`, return it unchanged without recursively rebuilding the whole expression.
- Add a structural scanner that detects lambda forms without invoking expensive expression reconstruction.
- Replace `map-flat` traversal with a more predictable list walk if `map-flat` interacts badly with nested expression siblings.
- Add a depth/node budget and return an explicit error rather than hanging.

For petta-memory's current statements, a no-lambda fast path should make `materialize-stmt-lambdas` an identity operation.

8.3 Priority 3: add a supported precompiled or bulk-add API

Many integrators will have already-normalized facts and rules. A supported API should let them load those facts while preserving whatever indexes and proof structures PeTTaChainer requires. The API could be named something like:

```
compileadd-checked  
compileadd-precompiled  
bulk-compileadd  
kb-add-compiled-proof-atoms
```

The key is not the name. The key is that callers should not be forced to use private low-level assertions that might bypass indexes. The API should state which invariants it maintains.

8.4 Priority 4: add timeout-safe add methods to the Python API

Mirror the query timeout design for `add_atom` and `add_atoms`. A safe method could return a structured object instead of raising raw exceptions:

```
AddResult(status, stage, elapsed_s, output, error, diagnostics)
```

This would make failures much easier for agent systems to report.

8.5 Priority 5: strengthen semantic validation

Keep `check_stmt` and `check_query`, but add optional deeper checks:

- proof-id symbol validity;
- rule variable safety;
- supported implication/premise/conclusion schemas;
- evidence-packet shape and nonnegative counts;

- context feature-list shape;
- distribution payload well-formedness;
- compiler-safety shape checks for known problematic nested forms.

This can be staged. A thin validator is useful, but a research integration path needs a “strict mode” validator too.

8.6 Priority 6: expose context search limits in outputs

Generated context results should always record which search method was used (pure enumeration, pure beam, Prolog beam), maximum depth, beam width, candidate count if available, selected guard, runner-up guard, score margin, and evidence cap/prior. This will make context projections more scientifically auditable.

9 Recommended petta-memory stance

For petta-memory, I recommend the following boundary until PeTTaChainer add/query is fixed:

1. Continue exporting promotion-eligible beliefs as checked STV proof statements and explicit-count EvidencePackets.
2. Keep the handoff cache labelled as input evidence, not inferred belief.
3. Do not claim PeTTaChainer inference over petta-memory until a bounded `compileadd` or supported equivalent passes on the tiny fixture.
4. Use PeTTaChainer semantics as a design reference for pi-PLN evidence and context semantics.
5. In parallel, prototype pi-PLN evidence/context wrappers around a more functional PLN chainer, currently `patham9/PLN`, for near-term experiments.

This is not a rejection of PeTTaChainer. It is a way to avoid letting one runtime bottleneck block the conceptual and integration progress.

10 Suggested regression tests for PeTTaChainer

10.1 Fast materializer identity tests

Add direct tests for `materialize-stmt-lambdas` on lambda-free terms:

```
(Requires MemoryTarget0 PLNReadyViews)
(STV 1.0 1.0)
(: b-profile-000 (Requires MemoryTarget0 PLNReadyViews) (STV 1.0 1.0))
(ProofEnvelope PayloadA (Requires TypeArgSentinel0 TypeArgSentinel1)
  (RightPayload 1.0 1.0))
```

Expected result: identity output under a small time bound.

10.2 Stage timing tests

For a tiny fact and a tiny one-premise implication, record time for each `compileadd` stage. Fail if any stage exceeds a conservative bound in CI, or at least mark it as a performance regression.

10.3 Batch-add diagnostics

Add a test that loads a list of atoms and returns per-atom status. The first failing atom should be identifiable without bisecting by hand.

10.4 Context projection reproducibility

For the bird/penguin example, record not only the final low-STV projection but also selected guard, support packets, score trace, and minimality report. Verify that irrelevant noise does not alter the selected guard unless the score margin is genuinely near zero.

11 Strengths and weaknesses table

Area	Strength	Weakness or risk
Semantic model	Context-indexed EC evidence, local STV projection, generated contexts, paraconsistent conflict handling.	Needs careful documentation of how raw evidence becomes EC counts; users may misuse STV confidence as EC evidence.
Python API	Small importable interface; query timeout support; contextual query result object.	Add path has no equivalent timeout isolation; temporary &self injection is fragile under concurrency/interruption.
Runtime layout	Clear split into chainer, context, configs, tests, benchmarks, and paper examples.	Some critical public behavior still depends on large fused MeTTa heads.
Compiler path	Named stages make repair possible.	<code>compileadd</code> fuses all stages; <code>materialize-stmt-lambdas</code> can block before compilation.
Validation	Useful top-level statement/query shape checks.	Not a strict semantic type checker; does not catch all rule/evidence/context risks.
Tests	Broad showcase, context, audit, verifier, and red-team coverage.	Needs small microtests for compiler/materializer shape regressions.
Packaging	Installable Python package with bundled MeTTa runtime data.	Heavy SWI/Janus/PeTTa dependency setup; CI/onboarding remain fragile.
Benchmarks	Covers dispatch, contexts, evidence merge, incident replay, and forensic packets.	Current failing petta-memory case needs more stage-isolated benchmarks than broad showcases.

12 Conclusion

PeTTaChainer is semantically promising and unusually close to what an OmegaClaw/petta-memory style experiential reasoner wants: context-sensitive, paraconsistent, proof-aware PLN over a MeTTa/PeTTa substrate. Its main weakness is that the operational add/compile path is not yet robust enough for dependable agent memory integration. The codebase should not be discarded; it should be stabilized around the materialization/compiler boundary and given a safe staged or precompiled load API.

Until then, petta-memory should preserve PeTTaChainer-compatible handoff artifacts and use the codebase as the semantic reference, while developing the near-term pi-PLN chainer path on a more functional substrate. This keeps the research direction intact without overstating current runtime maturity.

A Key source paths

- `pettachainer/pettachainer.py`: Python handler and API.
- `pettachainer/pln_validator.py`: statement/query shape checks.
- `pettachainer/LANGUAGE_SPEC.md`: user-facing language spec.
- `pettachainer/LLM_RULE_SPEC.md`: LLM-oriented rule construction spec.
- `pettachainer/metta/petta_chainer.metta`: MeTTa entry point, `materialize-stmt-lambdas`, `compileadd`.
- `pettachainer/metta/chainer/compile.metta`: compiler logic.
- `pettachainer/metta/chainer/compiled_query_runtime.metta`: query runtime.
- `pettachainer/metta/context/context_generation.metta`: generated contexts and EC-to-STV projection.
- `pettachainer/metta/context/context_from_kb.metta`: context evidence extraction from a KB space.
- `pettachainer/metta/context/context_inference_control.metta` and `context_inference_control_beam`: context-driven branch control.
- `pettachainer/metta/piPLN_paper_explained/`: runnable reading guide and demos.
- `pettachainer/benchmarks/`: benchmark and verification scripts.
- `tests/`: Python regression tests.

B Petta-memory artifacts informing this report

The following local artifacts and decisions informed the assessment. They are not all required reading for the PeTTaChainer author, but they record the integration evidence trail.

- `projects/petta-memory/DECISIONS.md`: PeTTaChainer integration, compileadd bottleneck, static-import side path, materializer gates, and pivot to `patham9/PLN` as a parallel functional chainer base.
- `projects/petta-memory/TASKS.md`: current tasks and latest status.
- `projects/petta-memory/repos/petta-memory/artifacts/`: structured JSON outputs from bounded profiling and materializer probes.
- Latest relevant materializer artifact: `pettachainer_materialize_four_field_adjacent_nested_arity_gat`